

Security Assessment Report

Browser Developer Tools and Postman-Style API Analysis

Evidence basis: Browser developer tools screenshots and extracted API observations

Scope: Client-visible third-party API requests to ipstack and numverify / APILayer

Executive Summary

The evidence indicates that third-party API calls to ipstack and numverify / APILayer were made directly from a browser context while embedding live access keys in GET query parameters. This is a significant security weakness because any user with browser developer tools can inspect, copy, and replay the credentials. In addition to credential exposure, several screenshots show broad response payloads and optional request parameters that appear to be controlled from the client side. Overall, the most important remediation is to move these integrations behind a backend service, rotate the exposed keys, restrict provider usage, and return only the minimum necessary data to the browser.

Methodology



Postman_API_Sec
urity_Analysis_Rep

- Browser developer tools review of visible network requests, query parameters, payloads, and console-level response inspection.
- Postman-style API analysis of endpoint structure, request parameters, authentication placement, and response fields based on the captured evidence.
- Correlation of repeated requests across screenshots to determine whether the issue was isolated or systematic.
- Qualitative severity assessment focused on credential exposure, potential abuse paths, data minimization, and privacy considerations.
- No destructive exploitation, credential misuse, or intrusive testing was performed as part of this report.

Scope and Limitations

- Scope includes 10 screenshots showing browser-based API requests and responses associated with ipstack and numverify / APILayer.
- Eight referenced text evidence files were listed in the source material, but they were effectively empty or unavailable during this document preparation workflow.
- This report therefore relies primarily on the screenshots and the extracted textual observations accompanying those screenshots.
- The assessment is suitable for internship documentation and focuses on secure design observations rather than active exploitation.

APIs and Endpoints Observed in the Evidence

The following endpoints were visible in the browser developer tools evidence. Keys have been redacted in this report for responsible handling.

- https://api.ipstack.com/160.39.144.19?access_key=<redacted>
- https://api.ipstack.com/134.201.250.155?access_key=<redacted>
- https://api.ipstack.com/134.201.250.155?fields=zip&access_key=<redacted>
- https://api.ipstack.com/72.229.28.185,110.174.165.78?access_key=<redacted>
- https://api.ipstack.com/134.201.250.155?language=ru&access_key=<redacted>
- https://api.ipstack.com/check?access_key=<redacted>
- https://apilayer.net/api/validate?number=14158586273&format=1&access_key=<redacted>
- https://apilayer.net/api/validate?number=14158586273&access_key=<redacted>

Findings Summary

ID	API / Area	Key weakness	Severity
F-01	ipstack	API key exposed in browser GET request	High
F-02	ipstack	Repeated client-side secret exposure	High
F-03	ipstack	Parameter tampering possible with exposed key	High
F-04	ipstack	Bulk query abuse possible with stolen key	High
F-05	ipstack	Optional parameters controllable from browser	High
F-06	ipstack	Client-side requester IP enrichment and key leakage	High
F-07	ipstack	Verbose data exposure to browser	High
F-08	numverify / APILayer	Phone-validation key exposed in URL	High
F-09	numverify / APILayer	Same secret reusable across request variants	High
F-10	numverify / APILayer	Full request/response visible in client debugging	High

Prioritized Remediation Plan

Priority	Action
Immediate	Rotate all exposed ipstack and API Layer keys and remove them from front-end code and requests.
Immediate	Route API calls through a backend service or API gateway under server-side secret management.
Short term	Add rate limits, request validation, and provider-side usage restrictions where available.
Short term	Minimize returned fields and prevent the browser from receiving unnecessary enrichment data.
Ongoing	Review logging, console output, privacy disclosures, and secure coding standards for future API integrations.

Evidence Review 1: ipstack standard IP lookup with exposed access key

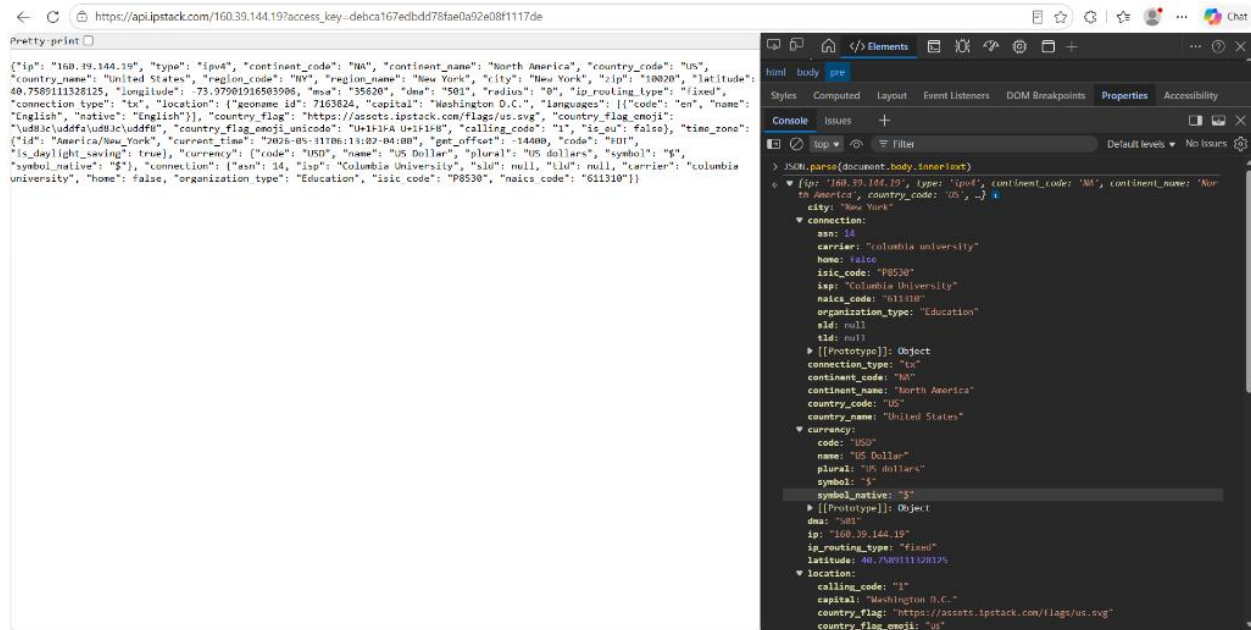


Figure 1. Screenshot evidence for F-01

Finding ID: F-01

API / Endpoint: https://api.ipstack.com/160.39.144.19?access_key=<redacted>

Severity: High

What the screenshot shows: The screenshot shows a browser developer tools request for a standard ipstack lookup against IP address 160.39.144.19. The response contains detailed geolocation and network metadata, including continent, country, region, city, ZIP code, latitude, longitude, ISP and connection-related fields.

Likely vulnerability or weakness: A live API access key is visible in the GET query string. This indicates the secret was embedded in client-side traffic, where it can be copied from browser developer tools, logs, proxies, browser history, or third-party monitoring systems.

Why it matters: An exposed key can enable unauthorized reuse of the paid API, quota exhaustion, unexpected billing, and unapproved access to the same third-party data service. It also increases the chance of abuse by anyone who can inspect the request.

Recommended remediation:

- Move third-party API calls behind a server-side proxy and store secrets only in backend environment variables or a secrets manager.
- Immediately rotate the exposed ipstack key and review historical usage for abuse.
- Apply domain, IP, quota, and usage restrictions where supported by the provider.
- Return only the minimum fields needed by the application instead of the full payload.

Evidence Review 2: ipstack lookup of California IP with credential leakage

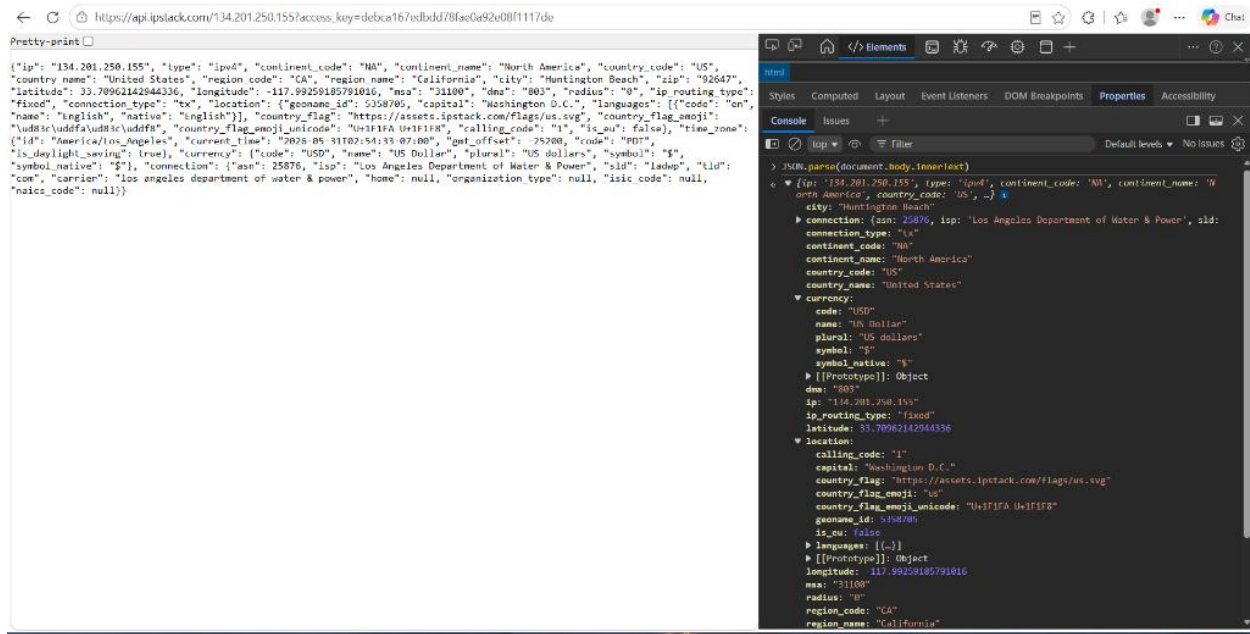


Figure 2. Screenshot evidence for F-02

Finding ID: F-02

API / Endpoint: https://api.ipstack.com/134.201.250.155?access_key=<redacted>

Severity: High

What the screenshot shows: This screenshot shows another standard ipstack geolocation lookup, this time for IP address 134.201.250.155. The returned data includes location details such as Huntington Beach, California, along with standard enrichment metadata.

Likely vulnerability or weakness: The authentication credential is again passed in plaintext within the URL. Repetition across multiple screenshots suggests the exposure is systemic rather than isolated.

Why it matters: Systemic client-side key exposure indicates any user or tester can replay requests. This creates operational and security risk across the entire integration, not just one sample query.

Recommended remediation:

- Remove all direct browser-to-provider requests that require secret keys.
- Centralize API integration in a backend service that authenticates the user and safely calls the third-party API.
- Instrument monitoring and alerts for abnormal third-party API consumption.
- Document secure integration patterns for all internship or project demos to avoid repeating the issue.

Evidence Review 3: ipstack field filtering request still exposes secret

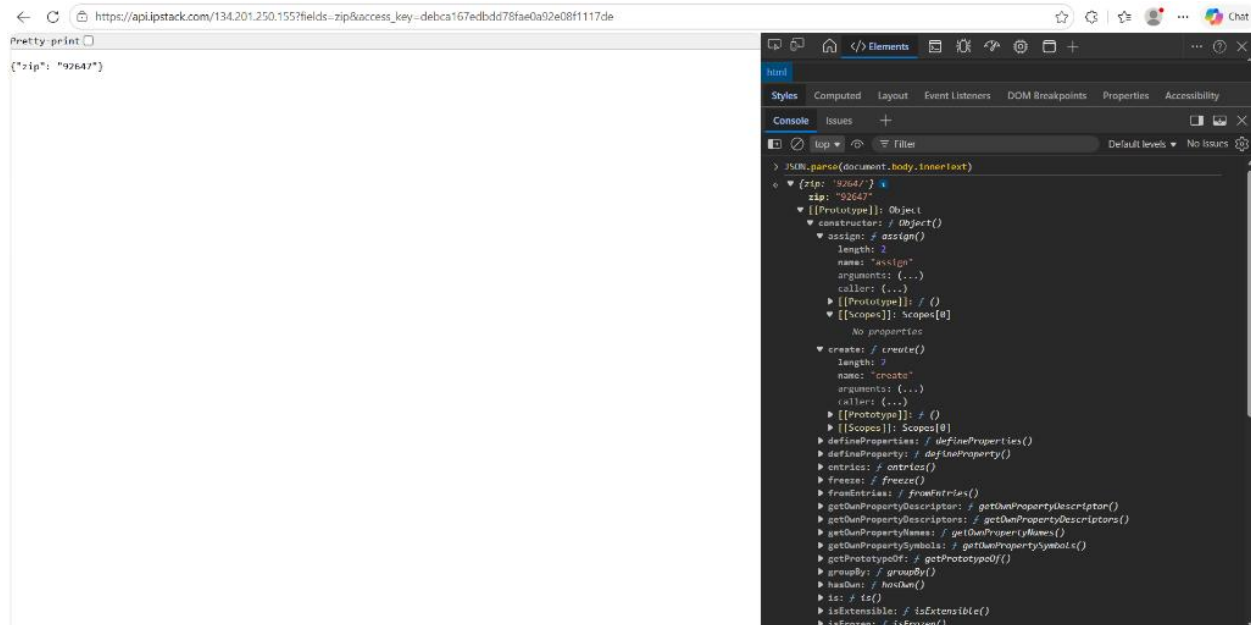


Figure 3. Screenshot evidence for F-03

Finding ID: F-03

API / Endpoint: `https://api.ipstack.com/134.201.250.155?fields=zip&access_key=<redacted>`

Severity: High

What the screenshot shows: The screenshot demonstrates use of the `fields` parameter to request only the ZIP code from ipstack. The response is a reduced object containing the requested field.

Likely vulnerability or weakness: Although the response is minimized, the API key is still exposed in the URL. The example also shows that request parameters can be manipulated from the client side, which may allow broader querying than originally intended.

Why it matters: Attackers can alter visible parameters and perform their own lookups using the stolen key. Even if the application UI limits what is shown, the backend provider remains directly reachable with the exposed credential.

Recommended remediation:

- Validate and enforce permitted parameters on a backend proxy instead of trusting browser-side controls.
- Implement allowlists for supported query fields and reject unnecessary enrichment requests.
- Continue using response minimization, but only after the authentication model is fixed.
- Rotate the credential because it has already been disclosed.

Evidence Review 4: ipstack bulk lookup enables larger-scale abuse if key is stolen

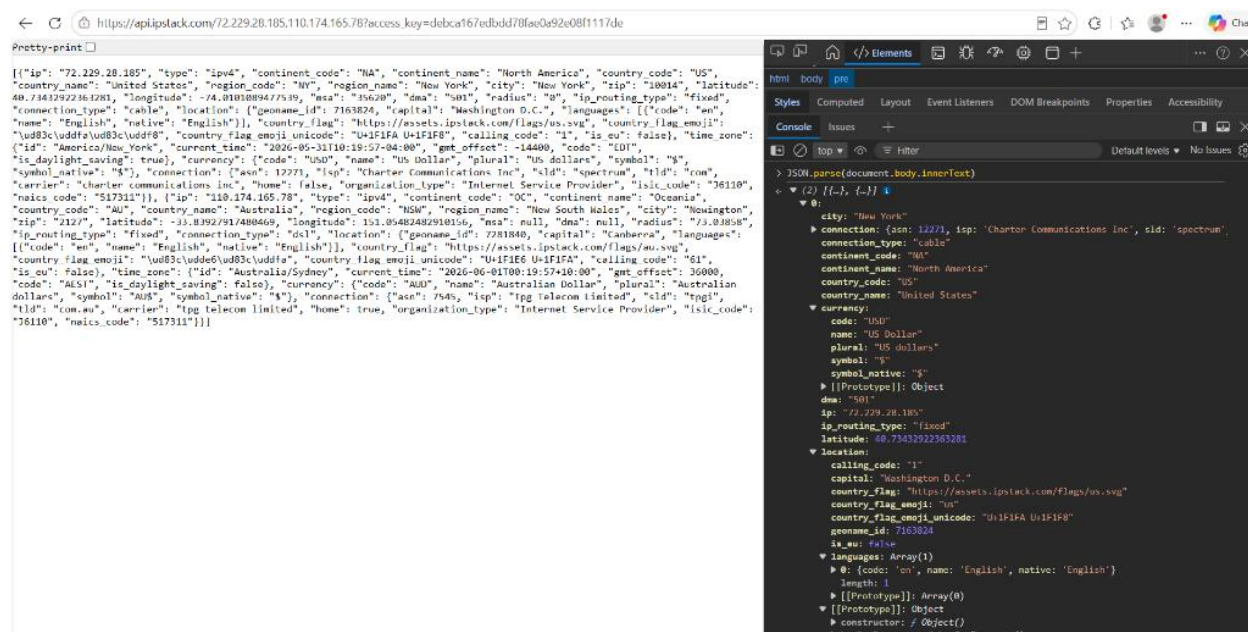


Figure 4. Screenshot evidence for F-04

Finding ID: F-04

API / Endpoint: `https://api.ipstack.com/72.229.28.185,110.174.165.78?access_key=<redacted>`

Severity: High

What the screenshot shows: The screenshot shows a bulk or multi-IP ipstack lookup where multiple IP addresses are submitted in a single request. The response contains an array of geolocation objects, one per IP address.

Likely vulnerability or weakness: The same plaintext access key is exposed, but the risk is amplified because bulk lookup functionality can support faster enumeration and larger-scale abuse when a key is copied.

Why it matters: A malicious user could automate multi-IP requests to consume quotas faster, build datasets, or generate higher provider costs than a single-record lookup would allow.

Recommended remediation:

- Disable direct client access and route batch operations through a controlled backend.
- Apply server-side rate limiting, request-size limits, and abuse detection for batch lookups.
- Consider whether bulk lookup is truly needed in the client workflow; if not, remove it.
- Rotate and scope the API key after remediation.

Evidence Review 5: ipstack localization parameter visible in browser traffic

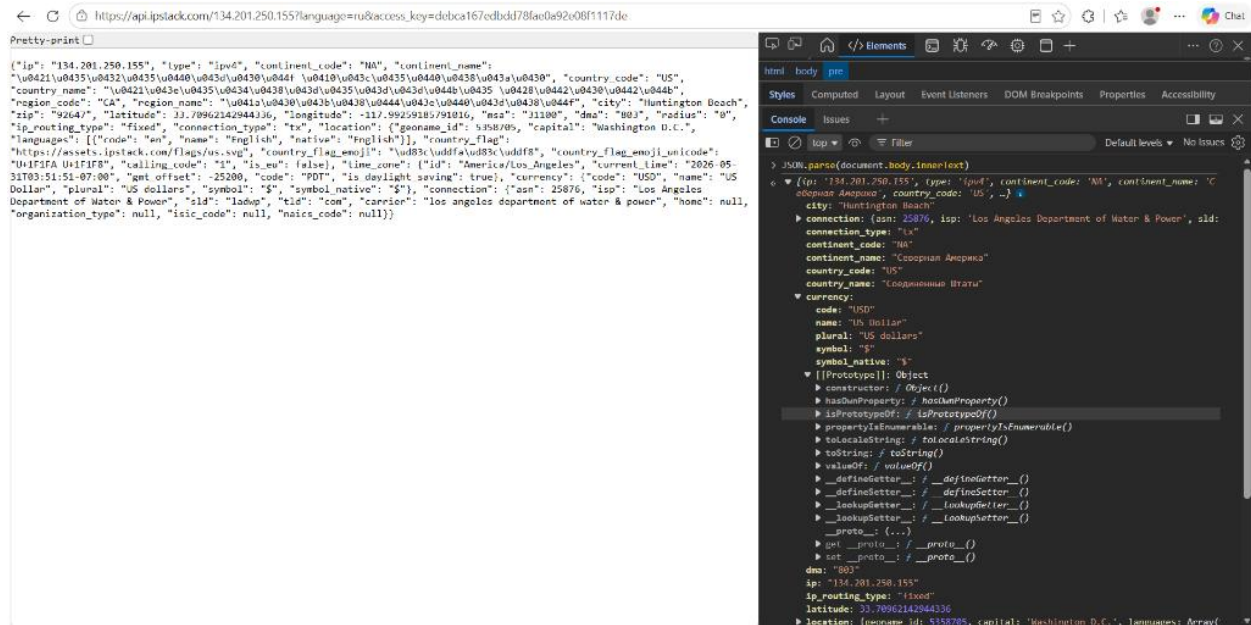


Figure 5. Screenshot evidence for F-05

Finding ID: F-05

API / Endpoint: https://api.ipstack.com/134.201.250.155?language=ru&access_key=<redacted>

Severity: High

What the screenshot shows: This evidence shows an ipstack request using the language=ru parameter. The response displays localized geolocation text in Russian, such as translated continent and country names.

Likely vulnerability or weakness: The screenshot confirms that optional provider features are being driven directly from the browser while still exposing the secret key. Parameter tampering is possible because the entire request structure is visible client-side.

Why it matters: An attacker with access to the key can vary request options, potentially increasing provider usage or querying data in ways the front-end workflow did not originally intend.

Recommended remediation:

- Move localization and third-party querying to a backend service that enforces accepted parameters.
- Expose only the localized fields needed by the application to the browser.
- Log and review parameter usage patterns server-side to detect misuse.
- Rotate the provider credential and remove it from front-end code and network calls.

Evidence Review 6: ipstack current requester lookup via /check endpoint

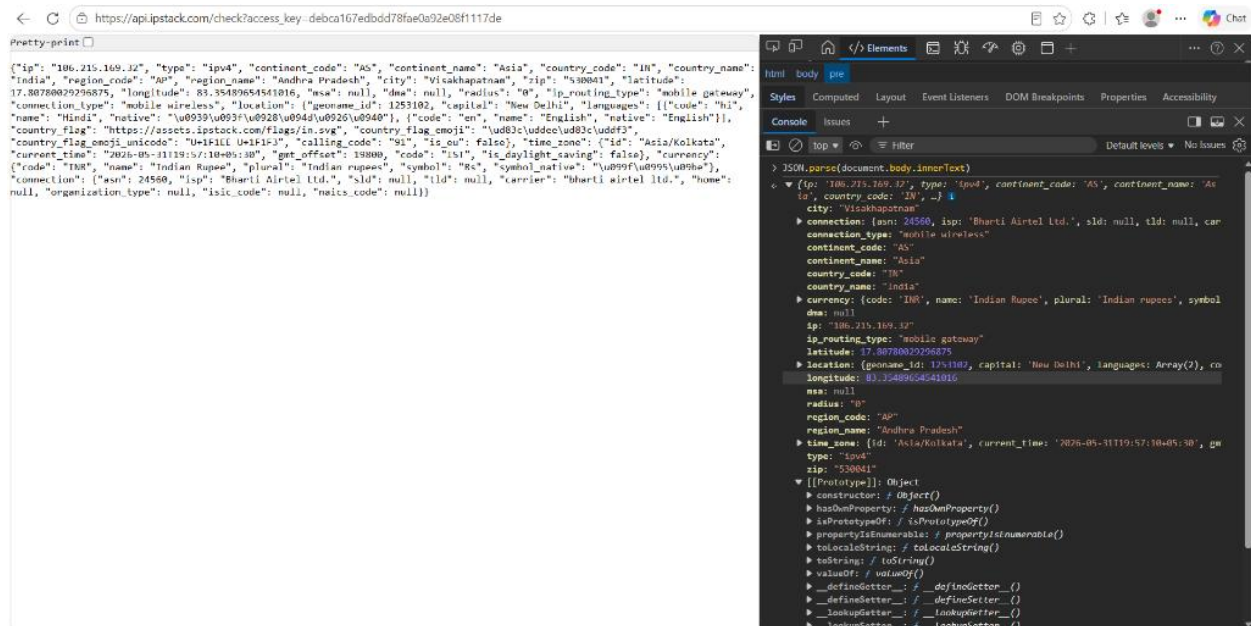


Figure 6. Screenshot evidence for F-06

Finding ID: F-06

API / Endpoint: `https://api.ipstack.com/check?access_key=<redacted>`

Severity: High

What the screenshot shows: The screenshot shows the ipstack `/check` endpoint, which returns geolocation data for the current requester IP address. This indicates the application can query visitor-related metadata directly from the browser session.

Likely vulnerability or weakness: The API key remains visible in the URL, and the design also implies direct sharing of visitor IP-derived information with a third-party provider from the client context.

Why it matters: Besides secret exposure, this pattern can raise privacy, compliance, and transparency concerns because visitor information is sent directly to an external service from the browser.

Recommended remediation:

- Proxy IP intelligence requests through the backend so the browser does not directly expose provider credentials.
- Review privacy disclosures and data-sharing requirements for transmitting requester IP information to third parties.
- Minimize or eliminate use of the `/check` endpoint unless clearly justified by business need.
- Rotate the existing key and verify that no unauthorized usage occurred.

Evidence Review 7: verbose ip metadata returned in client-visible response

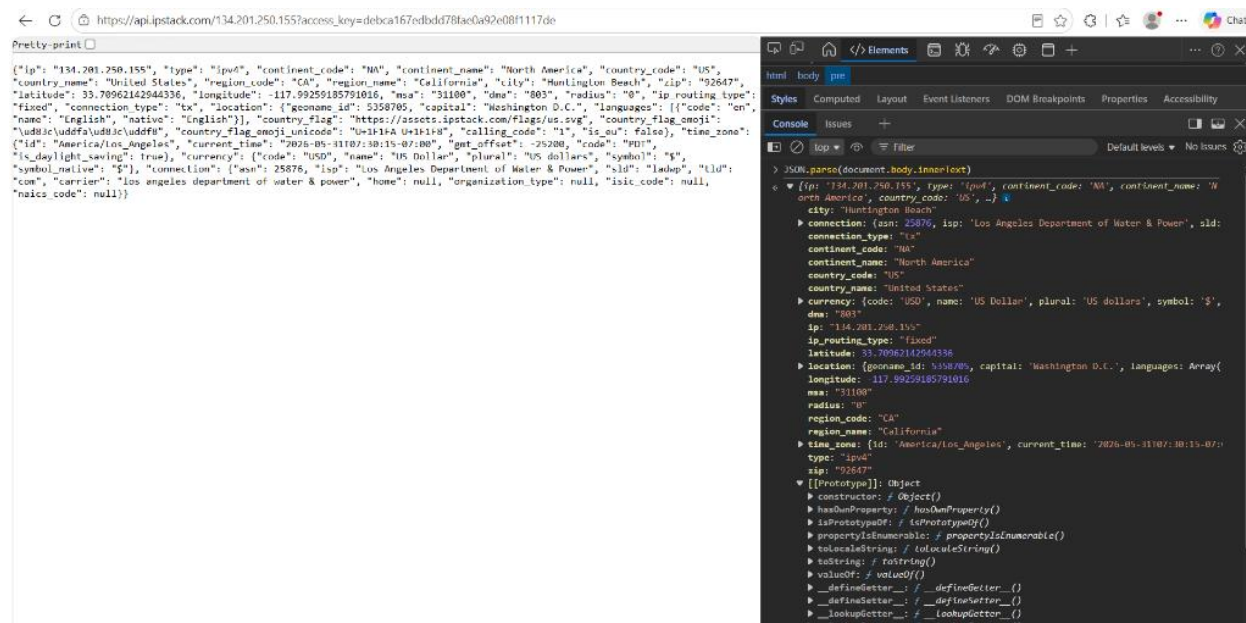


Figure 7. Screenshot evidence for F-07

Finding ID: F-07

API / Endpoint: `https://api.ipstack.com/134.201.250.155?access_key=<redacted>`

Severity: High

What the screenshot shows: This screenshot captures a developer tools payload view showing rich metadata such as time zone, currency, ISP, and connection information for the queried IP address.

Likely vulnerability or weakness: In addition to the exposed secret key, the response appears broader than many front-end use cases require. This suggests data overexposure and weak minimization practices.

Why it matters: Overly detailed responses increase unnecessary data exposure to the browser, enlarge the attack surface for scraping, and may create privacy or minimization concerns depending on project scope.

Recommended remediation:

- Request only fields that are essential for the actual business feature.
- Sanitize or reshape third-party responses on the backend before sending data to the browser.
- Review whether ISP, time zone, currency, and connection attributes are really required.
- Fix the key exposure issue before relying on response minimization alone.

Evidence Review 8: numverify formatted phone validation with exposed APIlayer key

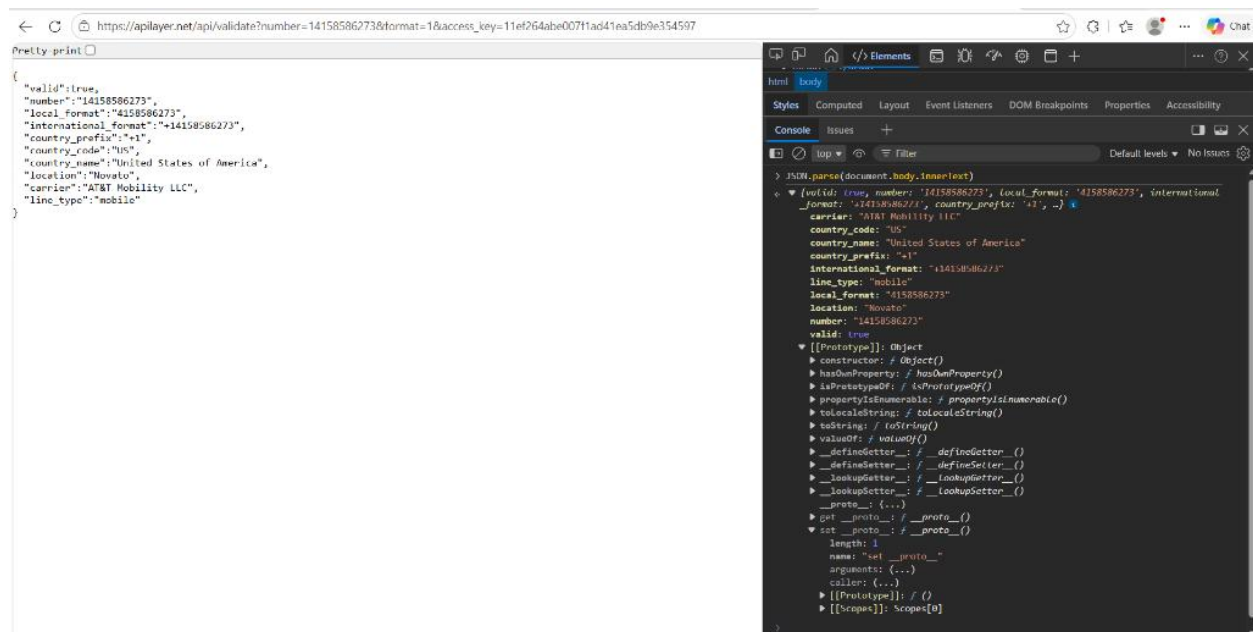


Figure 8. Screenshot evidence for F-08

Finding ID: F-08

API / Endpoint:

https://apilayer.net/api/validate?number=14158586273&format=1&access_key=<redacted>

Severity: High

What the screenshot shows: The screenshot shows a numverify phone validation request including the number, format option, and access key in the URL. The visible response fields include validity, local and international formats, country information, carrier, location, and line type.

Likely vulnerability or weakness: A high-value APIlayer credential is disclosed in the GET query string. The request also involves phone-number validation data, which can increase privacy sensitivity depending on the surrounding workflow.

Why it matters: An exposed phone validation key can be reused for unauthorized lookups, quota theft, or profiling attempts. If real user numbers are sent from the browser, privacy risk also increases.

Recommended remediation:

- Move phone validation to a backend service and never expose the APIlayer key in browser traffic.
- Mask, minimize, or tokenize phone data where feasible before logging or analytics collection.
- Restrict key usage and rotate the credential immediately.
- Assess whether client-side validation can be reduced to format checks, leaving external enrichment to the server.

Evidence Review 9: numverify default-format validation request reveals same secret

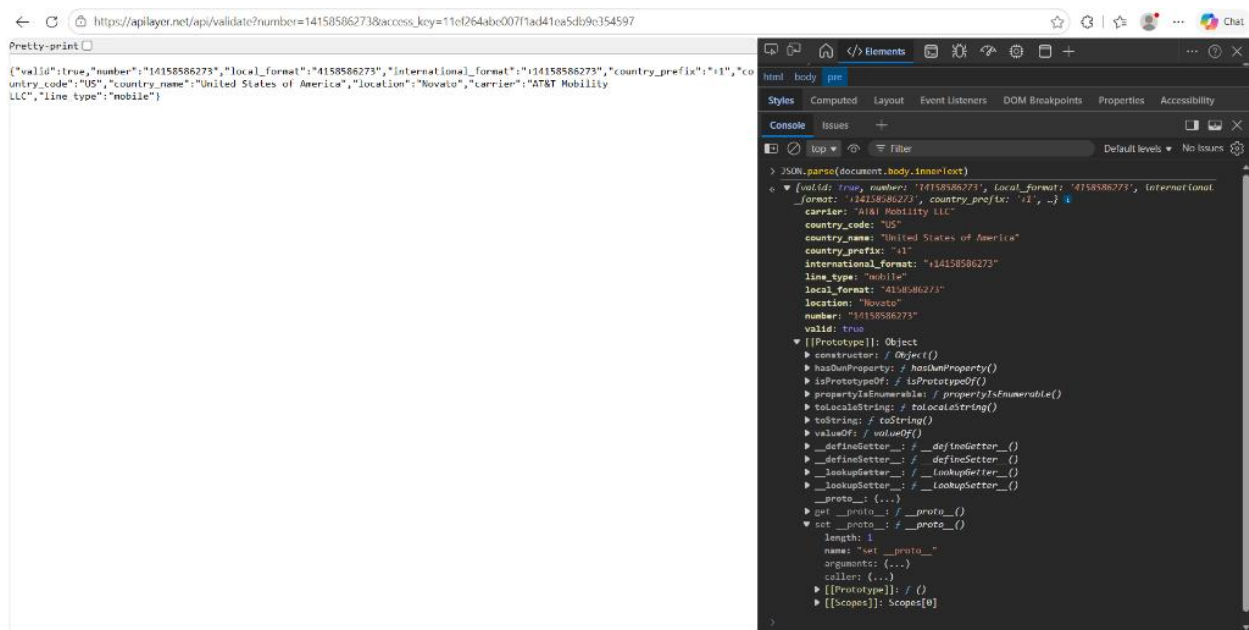


Figure 9. Screenshot evidence for F-09

Finding ID: F-09

API / Endpoint: `https://apilayer.net/api/validate?number=14158586273&access_key=<redacted>`

Severity: High

What the screenshot shows: This screenshot shows a similar numverify validation call without the format flag. The returned JSON still contains phone validation and carrier-related metadata.

Likely vulnerability or weakness: The screenshot confirms the secret exposure is not limited to one request variant. The key is reusable across multiple parameter combinations, making accidental or malicious replay easier.

Why it matters: Because the same secret supports multiple call patterns, an attacker can experiment with the provider API directly once the credential is copied from browser traffic.

Recommended remediation:

- Eliminate all direct browser calls to the validation API.
- Centralize parameter validation and outbound API calling logic on the backend.
- Rotate the credential and update all project documentation and sample code to use safe patterns.
- Monitor provider dashboards for suspicious bursts of validation traffic.

Evidence Review 10: developer-console inspection of numverify response structure

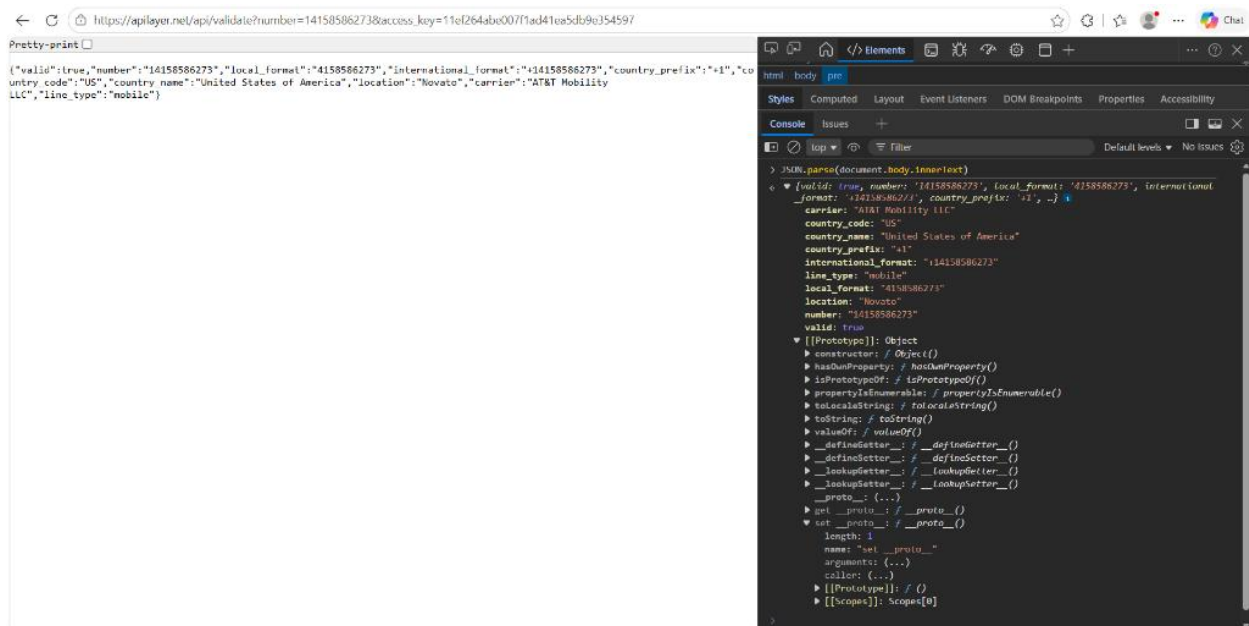


Figure 10. Screenshot evidence for F-10

Finding ID: F-10

API / Endpoint: `https://apilayer.net/api/validate?number=14158586273&access_key=<redacted>`

Severity: High

What the screenshot shows: The screenshot focuses on technical debugging or console inspection of the numverify response structure. It shows that the endpoint behavior and response fields are fully observable from the client-side development environment.

Likely vulnerability or weakness: The main issue remains secret exposure in the request URL, but the debugging context also demonstrates how easy it is for a tester or end user to inspect and repurpose the API interaction when it is implemented client-side.

Why it matters: Visibility of full request and response details reduces the effort required for unauthorized reuse, reverse engineering, and scripted abuse of the integration.

Recommended remediation:

- Keep debugging and provider integrations behind authenticated backend services.
- Remove secrets from front-end logic, rotate exposed keys, and reissue safer credentials.
- Review browser console logging and ensure no unnecessary sensitive details are emitted in production.
- Adopt a secure API integration checklist for future internship deliverables and demos.

Conclusion

Across all reviewed evidence, the dominant issue is the same: secret API credentials were exposed in client-side requests visible through browser developer tools. This design allows easy credential theft, request replay, quota abuse, and uncontrolled third-party API usage. Secondary concerns include excessive response detail and direct client control of optional API parameters. For an internship-ready remediation plan, the project should place all third-party API integrations behind a backend proxy, rotate every exposed key, enforce request allowlists and rate limits, minimize response data, and remove sensitive debugging output from production flows. If these actions are implemented, the application will be substantially safer and more professionally aligned with secure API integration practices.

Postman API Security Analysis Report

/

1. APIs Covered

This report analyzes two API collections shared through Postman:

- **IPStack API:** IP geolocation and requester IP lookup | Postman collection: <https://smartynandu18-7815593.postman.co/workspace/Nanda-Vardhan's-Workspace~f65b00ca-0dfa-48c1-8838-e0779b762401/collection/50703049-7a4759c7-8572-4f24-be0a-e29c90120d4a?action=share&source=copy-link&creator=50703049>
- **Numverify API:** Phone number validation and formatting lookup | Postman collection: <https://smartynandu18-7815593.postman.co/workspace/Nanda-Vardhan's-Workspace~f65b00ca-0dfa-48c1-8838-e0779b762401/collection/50703049-d1efd7dc-b412-4b6e-8130-14e773e005df?action=share&source=copy-link&creator=50703049>

2. Assessment Scope and Method

The screenshots show successful API requests executed in Postman for multiple IPStack and Numverify features. The purpose of this review is to explain what each screenshot demonstrates, identify security or design weaknesses visible in the evidence, and recommend practical remediation steps suitable for an internship report.

Important note: this is an observational security review based on screenshots and exported outputs, not a live penetration test. Issues such as exposed credentials and JSONP usage are directly visible. Risks such as XML parser abuse are conditional and should be treated as architectural concerns rather than confirmed exploitation.

3. Executive Summary of Findings

Finding	Severity	Why it matters	Recommended fix
API keys included in query strings	High	Credentials in URLs can leak through logs, screenshots, browser history, proxies, and referrers.	Move secrets to Authorization headers or a secure server-side secret store; rotate exposed keys.
Duplicate / misspelled credential parameters	Medium	Creates ambiguity, weakens request hygiene, and may cause accidental exposure or inconsistent validation.	Use a single validated credential parameter or header; reject unknown parameters.
JSONP callback responses	High	JSONP is legacy and increases XSS/abuse risk compared with modern CORS-based JSON APIs.	Disable JSONP and use standard JSON over HTTPS with proper CORS policy.
Sensitive request data sent via GET	Medium	Phone numbers and access keys appear in the URL, which is easier to capture and retain.	Use POST where appropriate for sensitive operations and never place secrets in URLs.
Overly broad data exposure in responses	Low to Medium	Geolocation, carrier, and routing data may exceed minimum business need.	Apply response minimization, fields filtering, and role-based access controls.
XML output support	Low to Medium	If backend XML parsers are unsafe, XML support can increase parsing risk.	Keep XML disabled unless required; harden parsers and prefer JSON.

4. Screenshot-by-Screenshot Explanation and Remediation

Screenshot 1 – IPStack Hostname Lookup

The screenshot displays the IPStack API interface. The URL bar shows a GET request to `https://api.ipstack.com/160.39.144.19?hostname=1`. The response is a 200 OK status with a JSON body. The JSON body contains the following fields:

```
{
  "ip": "160.39.144.19",
  "hostname": "160.39.144.19",
  "type": "ipv4",
  "continent_code": "NA",
  "continent_name": "North America",
  "country_code": "US",
  "country_name": "United States",
  "region_code": "NY",
  "region_name": "New York",
  "city": "New York",
  "zip": "10026",
  "latitude": 40.758911320125,
  "longitude": -73.979891916583986
}
```


API / Feature: IPStack | Standard IP Lookup | Hostname lookup feature

What the screenshot shows: A GET request to the IPStack endpoint retrieves geolocation details for an IP address and includes hostname resolution in the response.

Observed points

- The request uses query parameters including hostname=1 to enrich the output with hostname information.
- The response includes IP, hostname, continent, country, region, city, zip, and coordinates.
- The API key is supplied through the URL query string rather than a safer authorization mechanism.

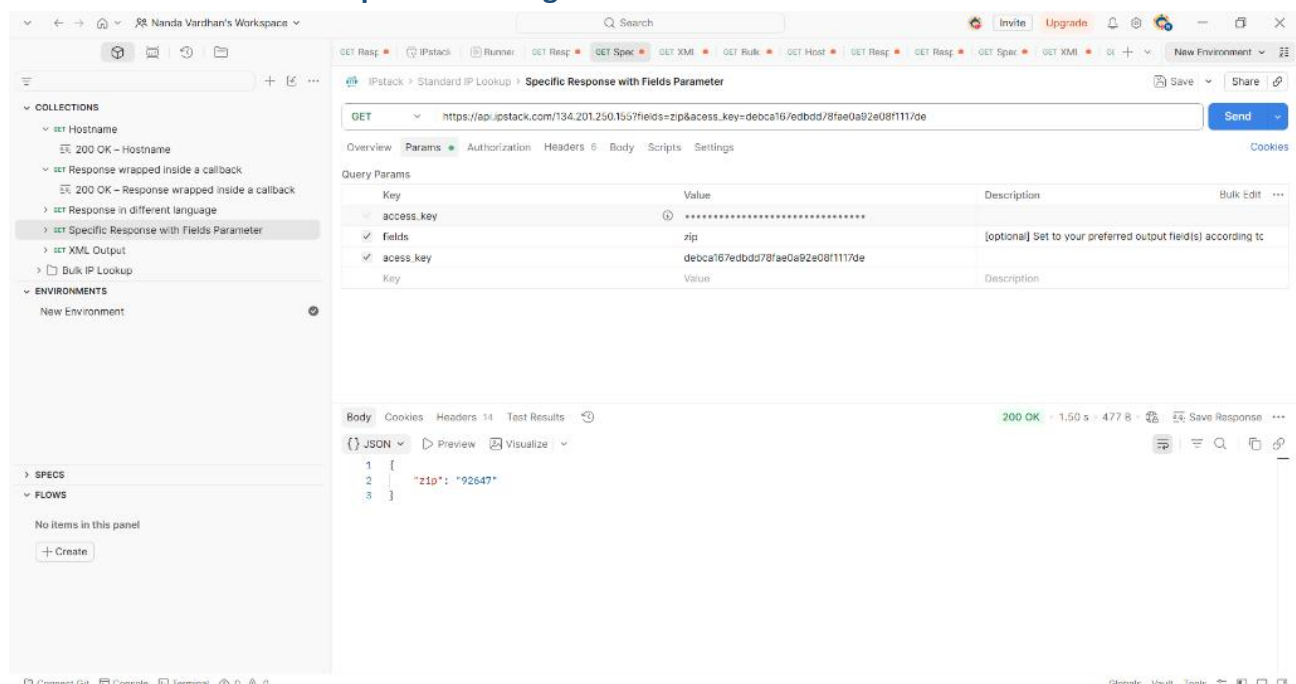
Security concern / weakness

- Credential exposure risk because URL-based secrets may be captured in logs, monitoring tools, screenshots, and browser history.
- Data minimization concern because the full response includes multiple fields that may not be needed for every use case.

Recommended remedies

1. Send credentials in an Authorization header or protect them server-side instead of exposing them in client-visible URLs.
2. Rotate the currently exposed key and remove it from shared screenshots or documentation.
3. Request only the minimum fields required for the business task.

Screenshot 2 – IPStack Response Filtering with Fields Parameter



API / Feature: IPStack | Specific response with fields parameter

What the screenshot shows: The request uses a fields parameter to limit the response to only the zip code.

Observed points

- This is a positive example of response minimization because only one field is returned.
- The screenshot still exposes a plaintext API key in the URL.

- A misspelled duplicate parameter appears as `access_key`, indicating weak request hygiene or environment misconfiguration.

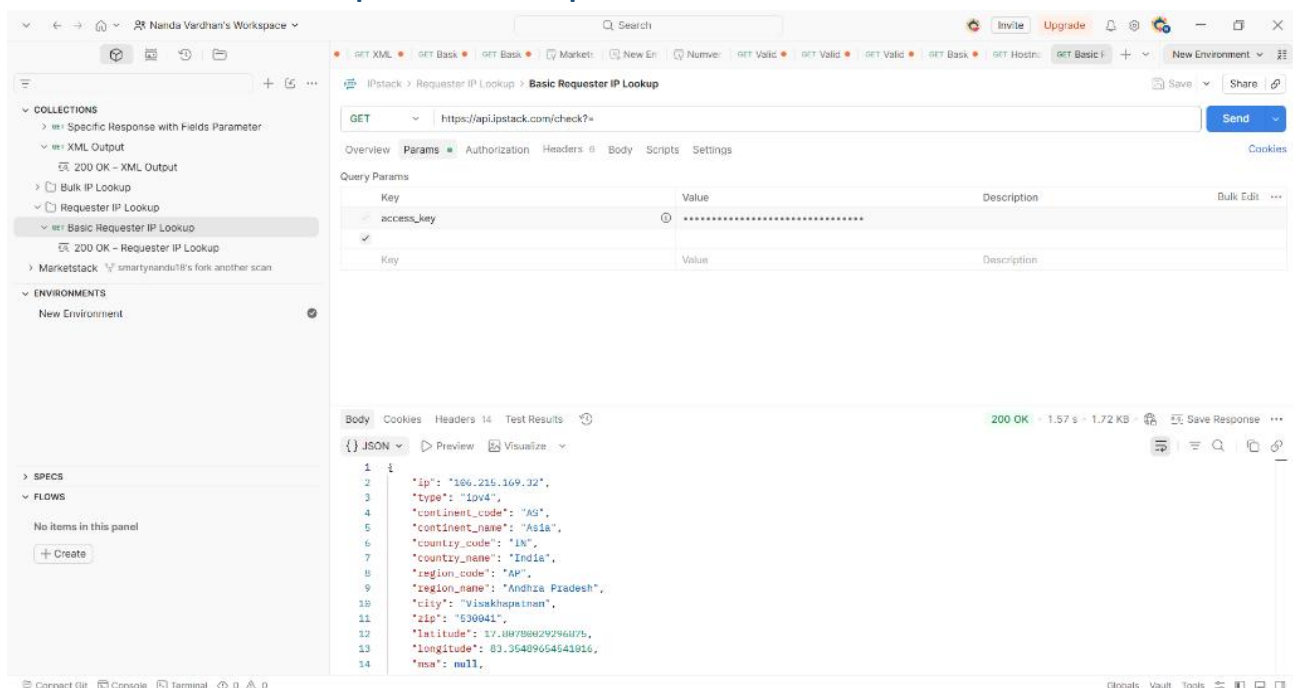
Security concern / weakness

- Even with minimized output, the secret remains exposed through the query string.
- Duplicate or misspelled parameters can cause unpredictable behavior and weaken request validation.

Recommended remedies

4. Keep the fields-based minimization approach because it reduces unnecessary exposure.
5. Fix parameter naming and validate requests strictly so invalid parameter names are rejected.
6. Store secrets in environment variables and inject them only in secure headers during runtime.

Screenshot 3 – IPStack Requester IP Lookup



API / Feature: IPStack | Requester IP Lookup

What the screenshot shows: The `/check` endpoint returns the requester's public IP and associated geolocation data.

Observed points

- The endpoint identifies the client IP automatically and returns regional data.
- The screenshot again shows `access_key` in query parameters.
- Requester IP lookup is useful operationally but still should follow least-privilege and least-exposure practices.

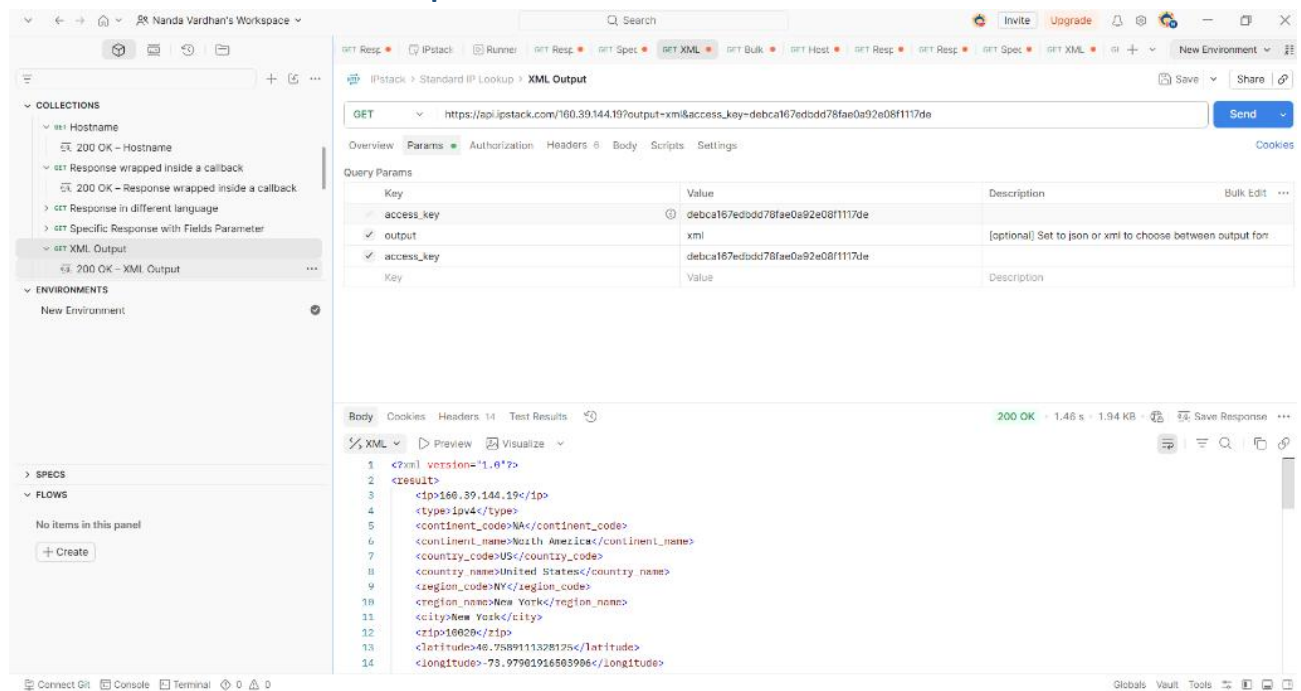
Security concern / weakness

- URL-based secret exposure remains the main visible issue.
- Requester IP data can be considered sensitive in some environments and should not be over-retained.

Recommended remedies

7. Use secure credential handling and rotate any exposed keys.
8. Define retention and masking rules for requester IP data in logs and analytics tools.
9. Restrict use of the endpoint to approved backend services where possible.

Screenshot 4 – IPStack XML Output



API / Feature: IPStack | XML output option

What the screenshot shows: The same IP lookup is requested with `output=xml`, and the response is returned in XML format.

Observed points

- The screenshot demonstrates alternate response serialization using XML.
- The API key is visible in the URL.
- XML support increases implementation complexity compared with a JSON-only API.

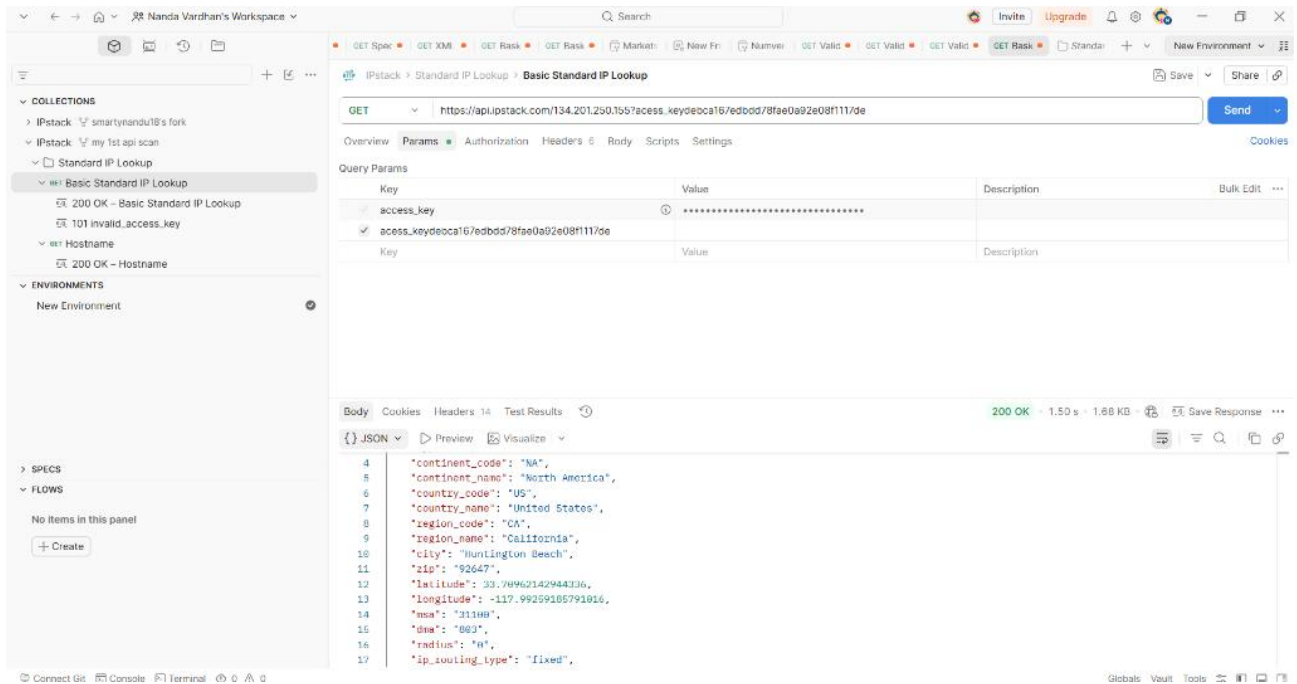
Security concern / weakness

- If any consuming system parses XML insecurely, XML-related parser attacks such as XXE become a design concern.
- Operational complexity increases when multiple output formats must be secured and tested equally.

Recommended remedies

10. Prefer JSON as the default and disable XML where it is not explicitly required.
11. If XML must be supported, disable external entities and harden all XML parsers.
12. Continue to remove secrets from query strings and rotate the exposed key.

Screenshot 5 – IPStack Basic Standard IP Lookup



API / Feature: IPStack | Basic standard IP lookup

What the screenshot shows: A standard IP geolocation request returns broad metadata including city, zip, latitude, longitude, and routing details.

Observed points

- The response is richer than a minimal lookup and includes routing-related fields.
- The credential is exposed in the URL and appears with a misspelled duplicate parameter.
- The screenshot suggests inconsistent request construction across the collection.

Security concern / weakness

- Excessive response detail may expose more information than necessary for simple geolocation use cases.
- Configuration inconsistency can create security blind spots in larger API collections.

Recommended remedies

13. Standardize all requests using collection variables and pre-request validation.
14. Return only the fields required by each use case.
15. Place secrets in headers or a secure backend and remove duplicate parameters.

Screenshot 6 – IPStack Response Wrapped Inside a Callback

The screenshot displays the IPStack API interface. The URL bar shows a GET request to `https://api.ipstack.com/134.201.250.155?callback=MY_FUNCTION&access_key=debca167edbdd78fae0a92e08f1117de&access_key=debca167edbdd78fae0a92e08f1117de`. The response is wrapped in a JavaScript callback function named `MY_FUNCTION`. The response body is a large JSON object containing detailed location and contextual metadata for the IP address 134.201.250.155, including country, region, city, and various flags.

Key	Value	Description
access_key	debca167edbdd78fae0a92e08f1117de	Response will be wrapped inside this function name
callback	MY_FUNCTION	
access_key	debca167edbdd78fae0a92e08f1117de	
access_key	debca167edbdd78fae0a92e08f1117de	

```
1 MY_FUNCTION({ "ip": "134.201.250.155", "type": "ipv4", "continent_code": "NA", "continent_name": "North America", "country_code": "US", "country_name": "United States", "region_code": "CA", "region_name": "California", "city": "Huntington Beach", "zip": "92647", "latitude": 33.78962142944336, "longitude": -117.99259105791816, "area": "31180", "dma": "803", "radius": "0", "ip_routing_type": "Fixed", "connection_type": "tx", "location": { "geoname_id": 5398795, "capital": "Washington D.C.", "languages": [ { "code": "en", "name": "English", "native": "English" } ], "country_flag": "https://assets.ipstack.com/flags/us.svg", "country_flag_emoji": "\ud83c\uddef\ud83c\uddfa", "country_flag_unicode": "U+1F1FA U+1F1F8", "calling_code": "1", "is_eu": false, "time_zone": { "id": "America/Los_Angeles", "current_time": "2026-05-31T07:55:08-07:00", "gmt_offset": -25200, "code": "PDT", "is_daylight_saving": true }, "currency": { "code": "USD", "name": "US Dollar", "plural": "US dollars", "symbol": "$", "symbol_native": "$", "connection": { "asn": 25876, "isp": "Los Angeles Department of Water & Power", "sld": "ladwp", "tld": "com", "carrier": "los angeles department of water & power", "hone": null, "organization_type": null, "isic_code": null, "naics_code": null } } })
```

API / Feature: IPStack | Response wrapped inside a callback (JSONP)

What the screenshot shows: The response is returned as JSONP using a callback function wrapper.

Observed points

- The request uses `callback=MY_FUNCTION`, producing JavaScript rather than plain JSON.
- The response contains a large amount of location and contextual metadata.
- The API key is exposed in the URL, and the credential parameter also appears duplicated/misspelled.

Security concern / weakness

- JSONP is a legacy pattern that can increase cross-site scripting and abuse risk compared with standard CORS-based JSON APIs.
- JavaScript-delivered data can be harder to secure consistently across clients.

Recommended remedies

16. Deprecate JSONP and migrate consumers to standard JSON with explicit CORS policies.
17. Use allowlisted origins and token-based authentication for browser access.
18. Rotate exposed secrets and sanitize all legacy integrations.

Screenshot 7 – Numverify Phone Validation with JSON Formatting Parameter

The screenshot shows a REST client interface with the following details:

- Request Method:** GET
- URL:** `https://apiplayer.net/api/validate?number=14158586273&callback=CALLBACK_FUNCTION&format=1&access_key=11ef264abe0071ad41ea5db9e354597`
- Query Params Table:**

Key	Value	Description
access_key	11ef264abe0071ad41ea5db9e354597	
number	14158586273	[Required] Phone number with prefix country code – 1 in it
callback	CALLBACK_FUNCTION	[Optional] Append any preferred JSONP callback function
format	1	[Optional] Setting up 1 enhances readability of the number
access key	11ef264abe0071ad41ea5db9e354597	

Response: 200 OK, 1.42 s, 711 B

```
{
  "CALLBACK_FUNCTION": [
    {
      "valid": true,
      "number": "14158586273",
      "local_format": "4158586273",
      "international_format": "+14158586273",
      "country_prefix": "+1",
      "country_code": "US",
      "country_name": "United States of America",
      "location": "Novato",
      "carrier": "AT&T Mobility LLC",
      "line_type": "mobile"
    }
  ]
}
```

API / Feature: Numverify | Validate phone number JSON formatting

What the screenshot shows: The Numverify validate endpoint checks a phone number and returns formatted metadata while also using callback and format parameters.

Observed points

- The response includes validity, local format, international format, country, location, carrier, and line type.
- The phone number and API key are both visible directly in the URL.
- The request mixes formatting behavior with callback-based delivery.

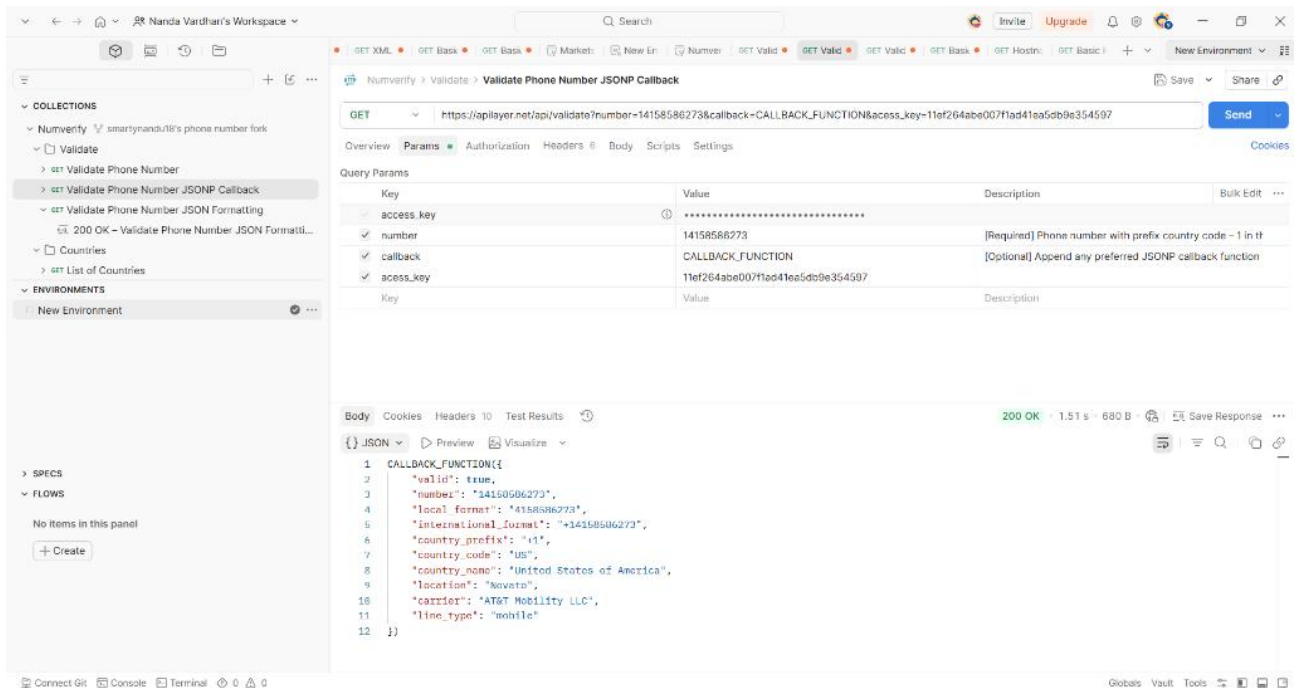
Security concern / weakness

- Phone numbers can be sensitive personal data; placing them in GET URLs increases leakage through logs and shared screenshots.
- JSONP-style behavior remains a legacy risk pattern.

Recommended remedies

19. Avoid placing phone numbers and secrets in URLs when possible; prefer safer transport patterns and backend mediation.
20. Use plain JSON over HTTPS instead of callback wrappers.
21. Redact sample phone numbers in shared internship or audit documentation unless explicitly approved.

Screenshot 8 – Numverify JSONP Callback Validation



API / Feature: Numverify | Validate phone number JSONP callback

What the screenshot shows: This request validates a phone number and wraps the result inside a callback function.

Observed points

- The response contains phone intelligence such as carrier and line type.
- The credential is visible in the URL with a duplicate misspelled parameter.
- The request demonstrates the same legacy JSONP pattern seen in the IPStack collection.

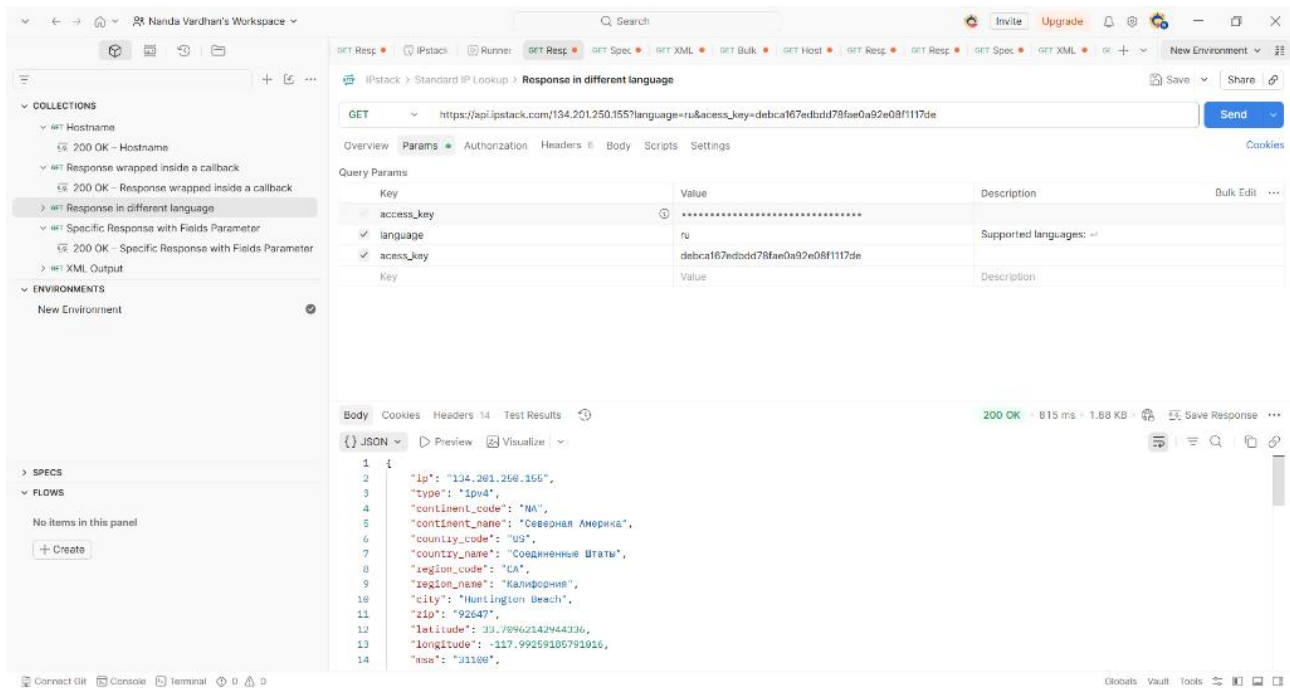
Security concern / weakness

- Exposed URL secrets and JSONP together create a poor security posture for browser-facing integrations.
- Input and response data may be logged broadly by intermediaries because the request is a GET request.

Recommended remedies

22. Remove JSONP support from client integrations and adopt standard CORS controls.
23. Move credentials to secure headers or a backend proxy service.
24. Introduce request templates or collection governance checks to prevent malformed parameters.

Screenshot 9 – IPStack Response in a Different Language



API / Feature: IPStack | Response in different language

What the screenshot shows: The IPStack response is localized using a language parameter and returned in Russian.

Observed points

- Localization is useful for user-facing products and international reporting.
- The key is still exposed in the URL, again via a duplicate or misspelled parameter.
- Localization increases the number of API behaviors that must be tested consistently.

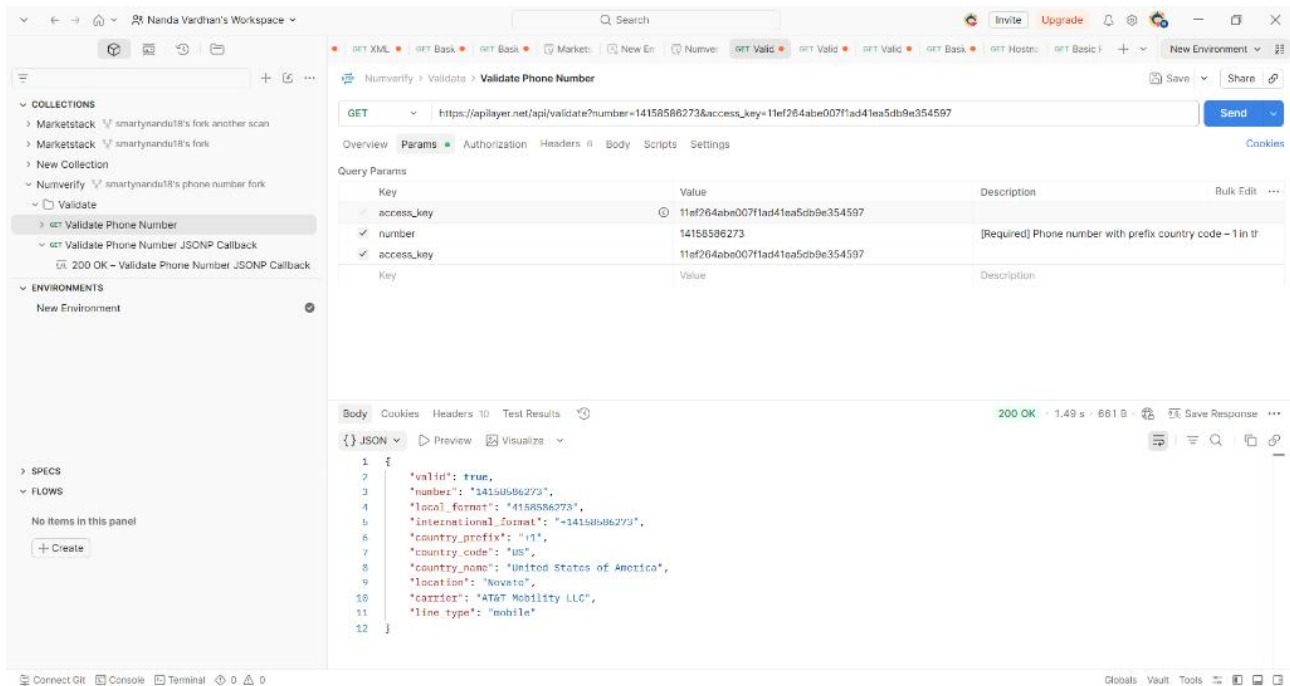
Security concern / weakness

- Although localization itself is not a vulnerability, every additional feature path increases testing surface.
- Credential leakage remains the direct security issue visible in the screenshot.

Recommended remedies

25. Keep localization testing separated from authentication handling so credentials are managed consistently.
26. Use centralized request templates and automated linting for collection variables.
27. Ensure responses remain minimized regardless of language option.

Screenshot 10 – Numverify Basic Phone Validation



API / Feature: Numverify | Validate phone number

What the screenshot shows: The Numverify validate endpoint returns a straightforward JSON validation result for a phone number.

Observed points

- The response confirms validity and returns formatting, carrier, country, location, and line type.
- The phone number and the API key are visible in the URL.
- The screenshot shows duplicate access_key handling in the request configuration.

Security concern / weakness

- Personally related phone metadata and API credentials can leak from logs, screenshots, or shared Postman environments.
- Improper environment sharing inside Postman can spread secrets across team members unintentionally.

Recommended remedies

28. Store secrets in protected environment variables and never export them in shared screenshots or collections.
29. Use role-based access to Postman workspaces and scrub sensitive values before sharing.
30. Apply logging minimization and data retention controls for phone-related queries.

5. Consolidated Remediation Roadmap

Immediate

- Rotate all exposed API keys visible in the screenshots and exported artifacts.
- Delete or replace any publicly shared screenshots, collections, and sample responses containing real secrets.

- Remove duplicate or misspelled credential parameters from every request in the Postman collections.

Short term

- Move authentication out of query strings and into secure headers or backend-managed secret handling.
- Disable JSONP and migrate all browser clients to standard JSON with CORS.
- Mask phone numbers, keys, and other sensitive values in logs and documentation.

Medium term

- Implement collection governance: naming standards, variable scoping, linting, and peer review.
- Adopt response minimization by default and request only fields required for each workflow.
- Document retention rules for IP and phone metadata.

Long term

- Introduce a secure API gateway or backend proxy to broker third-party API access.
- Use centralized secret management, key rotation, and environment segregation for dev, test, and production.
- Perform periodic API security reviews with automated scanning and manual validation.

6. Conclusion

The Postman collections successfully demonstrate the functionality of the IPStack and Numverify APIs, but the screenshots also reveal a repeated pattern of insecure request construction. The most important issue is the exposure of API keys in query strings across multiple requests. The second major issue is continued reliance on JSONP callback responses. Additional improvement areas include strict parameter validation, reduced response exposure, safer handling of phone-related data, and stronger collection governance.

For an internship submission, the screenshots provide good evidence of functional testing, but the report should clearly distinguish between observed weaknesses, likely risks, and confirmed vulnerabilities. That makes the analysis technically accurate and professionally credible.